Оглавление

- 1. Основы
  - Введение в SQL
  - Реляционная БД
  - Категории SQL команд
  - Основные команды psql
- 2. Типы данных
  - Встроенные типы
  - Приведение типов данных в SQL
- 3. Структура БД (DDL)
  - Команды для работы с БД
  - Команды для работы с таблицами
  - Первичный ключ (Primary Key)
  - Внешний ключ (Foreign Key)
  - Ограничения (Constraints)
  - Последовательности
  - Автоинкремент через serial в PostgreSQL
  - GENERATED AS IDENTITY
- 4. Проектирование БД
  - Процесс проектирования
  - Нормальные формы
- 5. Запросы — основы
  - Полная выборка
  - Арифметические операции
  - DISTINCT
  - WHERE, AND/OR
  - BETWEEN
  - IN / NOT IN
  - LIKE
  - ORDER BY
  - LIMIT
  - CHECK ON NULL
  - Условная логика — CASE, COALESCE, NULLIF
- 6. Изменение данных (DML)

    - INSERT
    - SELECT INTO, CREATE TABLE AS
    - UPDATE, DELETE, RETURNING
    - Импорт данных CSV в PostgreSQL
- 7. Массивы и пользовательские типы
    - Массивы
    - Пользовательские типы
- 8. Агрегация и группировка
    - COUNT
    - MIN, MAX, AVG, SUM
    - GROUP BY / HAVING
    - FILTER
    - GROUPING SETS, ROLLUP, CUBE
- 9. Объединение данных
    - JOIN
    - Псевдонимы (AS)
    - UNION, INTERSECT, EXCEPT
    - Подзапросы
    - Общие табличные выражения (CTE) в SQL
- 10. Оконные функции
    - Основные элементы оконных функций
    - Основные оконные функции
    - Окна и диапазоны
    - Границы окон
    - Порядок выполнения запроса
- 11. Объекты БД
    - Представления (View)
    - Функции в SQL и PL/pgSQL
    - Триггеры в PostgreSQL
- 12. Индексы и производительность
    - Устройство базы данных PostgreSQL
    - Структура таблиц PostgreSQL
    - Поддержка и очистка данных
    - Индексы и методы сканирования в PostgreSQL
    - Виды индексов в PostgreSQL
    - EXPLAIN и ANALYZE
    - Построение индексов — базовый и составной

    - Индексы по выражениям и их особенности
    - Индексы для полнотекстового поиска
- 13. Транзакции
    - Уровни изоляции
- 14. Безопасность
    - Роли и пользователи
    - Уровни безопасности
    - Создание и управление ролями
    - Лучшие практики
    - Управление доступом
    - Удаление ролей и прав
    - Важные моменты

---

1. Основы

Введение в SQL

SQL (Structured Query Language) — это стандартный язык для взаимодействия с реляционными базами данных. Он используется для определения, управления и извлечения данных из баз данных.

БД — это набор взаимосвязанных данных.

СУБД — это комплекс программных средств для управления данными (хранение, извлечение, оптимизация).

Типы СУБД: файл-серверные, клиент-серверные, встраиваемые.

Примеры клиент-серверных СУБД: MySQL, PostgreSQL, Oracle, MS SQL, MariaDB.

NoSQL — это подход к управлению данными, возникший для работы с нереляционными СУБД (например, MongoDB).

Основные операции, выполняемые с помощью реляционной алгебры: объединение, пересечение, вычитание, соединение.

Реляционная БД

Реляционная база данных — это база данных, основанная на реляционной модели данных. В такой базе данных данные хранятся в таблицах, и отношения между данными определяются с помощью ключей.

Основные характеристики:

- **Таблицы (отношения):** данные организованы в таблицы, состоящие из строк и столбцов.
- **Строки (записи):** каждая строка представляет отдельную запись.
- **Столбцы (атрибуты):** каждый столбец содержит данные определенного типа.
- **Ключи:** используются для идентификации записей и установления связей между таблицами.

Категории SQL команд

SQL-команды разделяются на основные категории:

- **DDL (Data Definition Language)** — язык определения данных:
  - Используется для определения структуры базы данных (схемы).
  - Основные команды: `CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`.
- **DML (Data Manipulation Language)** — язык манипулирования данными:
  - Используется для работы с данными внутри таблиц.
  - Основные команды: `SELECT`, `INSERT`, `UPDATE`, `DELETE`.
- **DCL (Data Control Language)** — язык управления доступом:
  - Используется для управления правами доступа к базе данных.
  - Основные команды: `GRANT`, `REVOKE`.
- **TCL (Transaction Control Language)** — язык управления транзакциями:
  - Управляет транзакциями в базе данных, контролирует фиксацию и откат изменений.
  - Основные команды: `COMMIT`, `ROLLBACK`, `SAVEPOINT`.

Основные команды psql

psql — консольное клиентское приложение для взаимодействия с сервером PostgreSQL.

Основные команды:

- `\c db_name` — подключение к базе данных.
- `\l` — список всех баз данных.
- `\dn` — список схем.
- `\dt` — список всех таблиц в текущей базе данных.
- `\d table_name` — описание структуры таблицы.
- `\dv` — список всех представлений базы данных.
- `\df` — список всех функций в базе данных.
- `\du` — список всех ролей в кластере.

- `\g` — повторить последнюю команду.
- `\i file_name` — выполнение команд из файла.
- `\?` — список всех команд psql.
- `\q` — выход из psql.

Использование:

- Добавьте путь к psql в переменную `PATH` или используйте SQL Shell (psql) для работы с командной строкой PostgreSQL.
- Пример запуска: `psql –U login`

---

2. Типы данных

Встроенные типы

Целочисленные типы

- `smallint` : 2 байта, диапазон от -32 768 до 32 767.
- `integer` : 4 байта, стандартный выбор, диапазон от -2 147 483 648 до 2 147 483 647.
- `bigint` : 8 байт, для больших чисел, диапазон от -9.22×10^{18} до 9.22×10^{18}.
- `serial` : автоинкрементируемое целое число (4 байта), диапазон от 1 до 2 147 483 647.
- `bigserial` : автоинкрементируемое большое целое число (8 байт).

Числовые с плавающей запятой

- `real` / `float4` : 4 байта, точность 6 десятичных знаков.
- `double precision` / `float8` : 8 байт, точность 15 десятичных знаков.
- `decimal` / `numeric` : переменный размер, точное значение с указанной пользователем точностью.

Символьные типы

- `char` : фиксированная длина строки, зависит от кодировки.
- `varchar` : строка переменной длины, зависит от кодировки.
- `text` : строка переменной длины без ограничений.

Логические типы

- `boolean` : логический тип, хранит значения `true` или `false` .

Временные типы

- `date` : 4 байта, хранит дату (диапазон от 4713 г. до н.э. до 294276 г. н.э.).
- `time` : 8 байт, хранит время (от 00:00:00 до 24:00:00).
- `timestamp` : 8 байт, хранит дату и время.
- `interval` : 16 байт, хранит разницу между метками времени.
- `timestamptz` : 8 байт, хранит метку времени с часовым поясом.

Приведение типов данных в SQL

1. **Неявное приведение** — автоматически преобразует совместимые типы, когда это возможно (например, при сложении `int` и `float`).
2. **Явное приведение** — выполняется при помощи `CAST` или `::` в PostgreSQL:
   - **Стандартный синтаксис:** `CAST(expression AS target_type)`
   - **PostgreSQL синтаксис:** `expression::target_type`

Примеры:

1. Приведение к `int`:

```sql
SELECT CAST(value AS int);
-- или
SELECT value::int;
```

2. Цепочка преобразований (`'123.45'::int` упадёт — строку сначала в `numeric`):

```sql
SELECT CAST('123.45' AS numeric)::int;
```

---

3. Структура БД (DDL)

DDL — это язык, который используется для определения и изменения структуры базы данных, включая таблицы и их столбцы.

Команды для работы с БД

Создание БД

```sql
CREATE DATABASE имя_базы_данных
[ WITH
    OWNER = владелец
    ENCODING = 'кодировка'
    LC_COLLATE = 'локаль_сортировки'        -- Порядок сортировки (локаль)
```

```
    LC_CTYPE = 'локаль_классификации'       -- Классификация символов
(локаль)
    TABLESPACE = имя_табличного_пространства
    CONNECTION LIMIT = ограничение_на_подключения  -- Ограничение на число
подключений
];
```

Удаление БД

```
DROP DATABASE [ IF EXISTS ] имя_базы_данных
[ FORCE ]; -- Принудительное удаление базы данных с активными подключениями
(PostgreSQL)
```

Изменение БД

```
ALTER DATABASE имя_базы_данных
[ SET параметр = значение ]
[ RENAME TO новое_имя ]
[ OWNER TO новый_владелец ]
[ SET ENCODING TO 'кодировка' ]; -- Изменение кодировки базы данных
(недоступно в PostgreSQL после создания)
```

Команды для работы с таблицами

- **CREATE TABLE:** создает новую таблицу.

  ```
  CREATE TABLE table_name (column_name data_type);
  ```

- **ALTER TABLE:** изменяет существующую таблицу (добавление, изменение или удаление столбцов).
  - **ADD COLUMN:** добавляет новый столбец в таблицу.
  - **RENAME TO:** переименовывает таблицу.
  - **RENAME COLUMN:** переименовывает столбец.
  - **ALTER COLUMN:** изменяет тип данных столбца или устанавливает значение по умолчанию.

  ```
  ALTER TABLE table_name
  ADD COLUMN column_name data_type;

  ALTER TABLE table_name
  RENAME TO new_table_name;

  ALTER TABLE table_name
  RENAME old_column_name TO new_column_name;
  ```

```
ALTER TABLE table_name
ALTER COLUMN column_name SET DATA TYPE new_data_type;
```

- **DROP TABLE:** удаляет таблицу из базы данных.

```
DROP TABLE table_name;
```

- **TRUNCATE TABLE:** удаляет все строки из таблицы, но сохраняет её структуру.

```
TRUNCATE TABLE table_name;
```

- **DROP COLUMN:** удаляет столбец из таблицы.

```
ALTER TABLE table_name
DROP COLUMN column_name;
```

Первичный ключ (Primary Key)

Первичный ключ (Primary Key) — это уникальный идентификатор для каждой строки в таблице. Он гарантирует уникальность значений и не допускает `NULL`. В каждой таблице может быть только один первичный ключ, и он может состоять из одного или нескольких столбцов (составной ключ).

Основные свойства первичного ключа:

- **Уникальность:** значения в поле первичного ключа должны быть уникальными.
- **Не допускает NULL:** все строки в таблице обязаны иметь значение в поле первичного ключа.
- **Индекс:** в большинстве СУБД (например, MySQL) при создании первичного ключа автоматически создается индекс для ускорения поиска данных по этому ключу.

```
CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    age INT
);
```

Составной первичный ключ: если уникальная идентификация строки возможна только при использовании комбинации нескольких столбцов, можно использовать составной первичный ключ.

Внешний ключ (Foreign Key)

Внешний ключ (Foreign Key) — это ограничение, которое устанавливает связь между двумя таблицами. Внешний ключ в одной таблице ссылается на первичный ключ или уникальный ключ другой таблицы. Основная цель внешнего ключа — поддержание ссылочной целостности данных.

Основные задачи внешнего ключа:

- **Целостность данных:** запись в одной таблице не может ссылаться на несуществующую запись в другой таблице.
- **Каскадные операции:** внешний ключ может быть настроен на автоматическое удаление или обновление записей при изменении связанной записи в другой таблице.

Пример внешнего ключа в таблице enrollment (зачисление):

```
CREATE TABLE enrollments (
    enrollment_id SERIAL PRIMARY KEY,
    student_id INT,
    course_id INT,
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);
```

Каскадное удаление/обновление: если нужно удалить или обновить записи, связанные с внешним ключом, можно использовать каскадное удаление или обновление.

```
CREATE TABLE enrollments (
    enrollment_id SERIAL PRIMARY KEY,
    student_id INT,
    course_id INT,
    FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE
CASCADE
);
```

В данном случае при удалении студента его записи в таблице `enrollments` также будут удалены.

Ограничения (Constraints)

Ограничения — это правила, которые накладываются на данные в таблице для обеспечения их целостности и правильности.

- **PRIMARY KEY:** гарантирует уникальность данных и отсутствие `NULL`.

- **UNIQUE:** обеспечивает уникальные значения в столбце, но допускает `NULL` (если не указано ограничение `NOT NULL`).
- **NOT NULL:** запрещает наличие пустых значений.
- **FOREIGN KEY:** устанавливает связь между таблицами, обеспечивая ссылочную целостность данных.
- **CHECK / DEFAULT**

```
CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
    email VARCHAR(100) UNIQUE NOT NULL
);
```

CHECK

- **CHECK** — ограничение, которое проверяет значения в столбце на соответствие заданным условиям.
- Применяется при вставке и обновлении данных.
- Используется для обеспечения бизнес-логики на уровне базы данных (например, возраст сотрудников должен быть от 18 до 65 лет).

```
CREATE TABLE employees (
    employee_id SERIAL PRIMARY KEY,
    name VARCHAR,
    age INT CHECK (age >= 18 AND age <= 65)
);
```

DEFAULT

- **DEFAULT** — ограничение, которое задает значение по умолчанию для столбца, если оно не указано при вставке данных.
- Автоматически подставляет значение, если его не указали.
- Помогает избегать `NULL` значений и облегчает работу с типичными данными.

```
CREATE TABLE products (
    product_id SERIAL PRIMARY KEY,
    product_name VARCHAR,
    quantity INT DEFAULT 0  -- Значение по умолчанию
);
```

Последовательности

Последовательности — это объекты в реляционных базах данных, предназначенные для генерации уникальных числовых значений, обычно используемых в качестве автоинкрементных значений для первичных ключей.

Создание последовательности:

```
CREATE SEQUENCE имя_последовательности
[INCREMENT шаг] [MINVALUE мин] [MAXVALUE макс] [START начальное];
```

- `nextval()` — возвращает следующее значение последовательности:
  ```
  SELECT nextval('имя_последовательности');
  ```
- `currval()` — возвращает текущее значение последовательности:
  ```
  SELECT currval('имя_последовательности');
  ```
- `setval()` — устанавливает новое значение для последовательности:
  ```
  SELECT setval('имя_последовательности', новое_значение, [is_called]);
  ```
- `ALTER SEQUENCE` — изменяет параметры последовательности:
  ```
  ALTER SEQUENCE имя_последовательности
  [RESTART [WITH] новое_значение] [INCREMENT шаг] [MINVALUE мин] [MAXVALUE
  макс];
  ```
- `DROP SEQUENCE` — удаляет последовательность:
  ```
  DROP SEQUENCE имя_последовательности;
  ```

Автоинкремент через serial в PostgreSQL

serial — это тип данных, автоматически создающий последовательность для генерации уникальных значений, обычно для первичных ключей.

```
CREATE TABLE book (
    book_id serial PRIMARY KEY,
    title text NOT NULL,
    isbn varchar(32) NOT NULL,
    publisher_id int NOT NULL
);
```

Под капотом `serial` создаёт последовательность и автоматически использует её для генерации значений. Это эквивалентно созданию последовательности вручную:

```
CREATE SEQUENCE IF NOT EXISTS book_book_id_seq START WITH 1;
ALTER TABLE book ALTER COLUMN book_id SET DEFAULT
nextval('book_book_id_seq');
```

Проблема с `serial`: если вставить значение вручную в поле `book_id`, последовательность не обновляется, что может вызвать ошибки уникальности.

GENERATED AS IDENTITY

GENERATED AS IDENTITY — это современный способ создания автоинкрементных полей, который заменяет `serial` в PostgreSQL 10+. Он устраняет многие проблемы, связанные с `serial`, и предоставляет больше контроля.

```
CREATE TABLE book (
    book_id int GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    title text NOT NULL,
    isbn varchar(32) NOT NULL,
    publisher_id int NOT NULL
);
```

- **GENERATED ALWAYS AS IDENTITY:** предотвращает ручную вставку значений, исключая конфликты.
- **GENERATED BY DEFAULT AS IDENTITY:** позволяет вставлять значения вручную, но по умолчанию использует автоинкремент.
- Для сброса последовательности:
  ```
  ALTER TABLE book ALTER COLUMN book_id RESTART WITH 10;
  ```
- Для временной ручной вставки значения в поле, которое сгенерировано с помощью `GENERATED ALWAYS AS IDENTITY`:
  ```
  INSERT INTO book OVERRIDING SYSTEM VALUE VALUES (3, 'title', 'isbn', 1);
  ```

---

4. Проектирование БД

Процесс проектирования

1. **Анализ требований:** определение потребностей предметной области и создание концептуальной схемы.
2. **Логическое проектирование:** детализирует модель данных, включая ключи, типы

данных и логические ограничения.
3. **Физическое проектирование:** выбор СУБД, определение индексов, типов данных и ограничений на доступ.

Базовые рекомендации:

- Таблицы представляют объекты, события или абстракции.
- Каждое поле должно содержать только одно значение.
- Избегайте хранения вычисляемых данных и полей с несколькими логическими частями.
- Нормализуйте данные для устранения избыточности и улучшения согласованности данных.
- Иногда можно нарушать правила для оптимизации производительности (например, денормализация).

Нормализация — это процесс организации структуры базы данных с целью уменьшения избыточности данных и обеспечения их целостности.

Нормальные формы

1НФ (Первая нормальная форма):

- Нет дубликатов строк.
- Все атрибуты простых типов.
- Все значения скалярные.

2НФ (Вторая нормальная форма):

- Таблица должна удовлетворять 1НФ.
- Каждый неключевой атрибут полностью зависит от первичного ключа.

3НФ (Третья нормальная форма):

- Таблица удовлетворяет 2НФ.
- Нет зависимости одних неключевых атрибутов от других — все атрибуты зависят только от первичного ключа.

---

5. Запросы — основы

Полная выборка

Выбирает все столбцы или указанные столбцы из таблицы.

```
SELECT * | столбец1, столбец2.. FROM имя_таблицы;
```

Арифметические операции

Выполняет математические операции над значениями столбцов.

```
SELECT столбец1 * столбец2 [AS новое_имя] FROM имя_таблицы;
```

DISTINCT

Убирает дубликаты, выбирает только уникальные значения.

```
SELECT DISTINCT столбец1[, столбец2, ...] FROM имя_таблицы;
```

WHERE, AND/OR

Фильтрация данных по условиям.

Операторы для условий WHERE:

- Сравнение: `=`, `!=`, `<`, `>`, `<=`, `>=`.
- Логические операторы: `AND`, `OR`, `NOT`.
- Специальные операторы:
- `LIKE`: для шаблонных совпадений.

```
SELECT * FROM имя_таблицы WHERE условие [AND/OR условие …];
```

BETWEEN

Выбирает строки, где значения находятся в указанном диапазоне.

```
SELECT * FROM имя_таблицы WHERE столбец BETWEEN значение1 AND значение2;
```

IN / NOT IN

Проверяет, содержится ли значение в списке.

```
SELECT * FROM имя_таблицы WHERE столбец [NOT] IN (значение1, значение2,
```

```
...);
```

LIKE

Используется для поиска строк по шаблону.

- `%`: 0, 1 или более символов.
- `_`: один любой символ.

```
SELECT * FROM имя_таблицы WHERE столбец LIKE 'шаблон';
```

ORDER BY

Сортирует результат по одному или нескольким столбцам в порядке возрастания или убывания.

```
SELECT * FROM имя_таблицы [...] ORDER BY столбец1 [ASC/DESC][, столбец2
[ASC/DESC]…];
```

LIMIT

Ограничивает количество возвращаемых строк.

```
SELECT * FROM имя_таблицы LIMIT количество_строк;
```

CHECK ON NULL

Проверяет наличие или отсутствие NULL значений.

```
SELECT столбец1, столбец2
FROM имя_таблицы
WHERE столбец1 IS [NOT] NULL;
```

Условная логика — CASE, COALESCE, NULLIF

CASE WHEN

CASE — это оператор в SQL, позволяющий выполнять условные логические проверки и возвращать различные значения в зависимости от выполнения условий.

```sql
SELECT product_name, unit_price,
    CASE
        WHEN unit_price >= 100 THEN 'Expensive'
        WHEN unit_price >= 50 THEN 'Average'
        ELSE 'Low Price'
    END AS price_category
FROM products;
```

- **CASE WHEN** проверяет условия последовательно, возвращает результат первого истинного условия.
- Если ни одно условие не выполнено, используется значение **ELSE**.

COALESCE и NULLIF

- **COALESCE(arg1, arg2, ...)** возвращает первый не-NULL аргумент, что удобно для подстановки значений по умолчанию.
- **NULLIF(arg1, arg2)** возвращает `NULL`, если аргументы равны, иначе возвращает первый аргумент.

Пример использования:

```sql
SELECT contact_name, COALESCE(city, 'Unknown') AS city
FROM customers;

SELECT COALESCE(NULLIF(city, ''), 'Unknown') AS city
FROM customers;
```

- **COALESCE** позволяет подставить значение по умолчанию вместо `NULL`.
- **NULLIF** используется для проверки на равенство и замены значений.

---

6. Изменение данных (DML)

INSERT

Команда `INSERT` используется для добавления новых строк в таблицу.

```sql
INSERT INTO table_name [(column1, column2...)]
VALUES (value1, value2)[, (value3, value4), ...];
```

ON CONFLICT (UPSERT)

Что делать, если вставка нарушает `UNIQUE` / `PRIMARY KEY` : пропустить или обновить существующую строку.

```
INSERT INTO products (product_id, product_name, quantity)
VALUES (1, 'Chai', 10)
ON CONFLICT (product_id) DO UPDATE
SET quantity = products.quantity + EXCLUDED.quantity;  -- EXCLUDED = строка,
которую пытались вставить

INSERT INTO products (product_id, product_name) VALUES (1, 'Chai')
ON CONFLICT (product_id) DO NOTHING;
```

SELECT INTO, CREATE TABLE AS

Создает новую таблицу на основе данных из другой таблицы.

```
SELECT * INTO new_table
FROM existing_table [WHERE condition];
```

Но по рекомендации лучше:

```
CREATE TABLE new_table AS
SELECT * FROM existing_table;
```

UPDATE, DELETE, RETURNING

UPDATE

Обновляет данные в таблице.

```
UPDATE table_name
SET column1 = value1, column2 = value2
WHERE condition;
```

Важно: если не использовать `WHERE` , обновятся все строки.

DELETE

Удаляет строки из таблицы.

```
DELETE FROM table_name WHERE condition;
```

Без `WHERE` удаляются все строки.

RETURNING

Возвращает измененные или удаленные данные.

```
UPDATE table_name SET column = value WHERE condition RETURNING *;
```

Импорт данных CSV в PostgreSQL

Основные шаги:

1. **Создайте таблицу** с полями, соответствующими структуре CSV.
2. **Подключитесь к базе данных** через psql и выполните команду импорта.

Шаблон команды для импорта CSV:

```
\copy table_name(column1, column2, ...) FROM 'csv_file_path' DELIMITER ','
CSV HEADER;
```

- `table_name` — имя таблицы, в которую производится импорт.
- `csv_file_path` — путь к файлу CSV.
- `DELIMITER ','` — указывает разделитель значений (запятая по умолчанию).
- `CSV HEADER` — обозначает, что первая строка файла содержит названия колонок.

`\copy` читает файл на стороне клиента (psql); `COPY` без слеша — на стороне сервера и требует прав на файловую систему сервера.

---

7. Массивы и пользовательские типы

Массивы

Массив — коллекция данных одного типа (например, 10 чисел типа `int`, строки типа `varchar`). В PostgreSQL массивы могут быть одномерными и многомерными (двумерные используются редко, трёхмерные и более — почти не встречаются).

Объявление массивов

- SQL стандарт: `temp int ARRAY`, `temp int ARRAY[4]`
- PostgreSQL: `temp int[]`, `temp int[8]`
    - Указание размера служит только для документации, на сам массив не влияет.

Инициализация массивов

- Примеры:
  - `'{1, 2, 3}'`
  - `'{{1,2,3}, {1,2,3}}'`
  - `ARRAY['a', 'b', 'c']`

Обращение и операции с массивами

- Доступ к элементу: `arr[index]` (индексация с 1)
- Срезы:
  - `arr[1:3]` — с 1 по 3 элемент
  - `arr[:4]` — с 1 по 4 элемент
  - `arr[2:]` — со 2 до последнего элемента
- Получение информации:
  - `array_dims(arr)` — размерность массива
  - `array_length(arr, dim)` — длина массива по указанной размерности

Заметка: массивы — не множества, и частый их поиск может указывать на неподходящую структуру базы данных. В таких случаях лучше использовать отдельные таблицы для элементов массива.

Работа с массивами

1. **Создание массива:**

```
CREATE TABLE table_name (
    array_column data_type[]
);
```

2. **Инициализация:**
   - Одномерный: `ARRAY['val1', 'val2']`
   - Многомерный: `ARRAY[['val1', 'val2'], ['val3', 'val4']]`
3. **Доступ и срезы:**
   - Элемент: `array_column[1]`
   - Срез: `array_column[1:3]`
4. **Информация о массиве:**
   - Размерность: `array_dims(array_column)`
   - Длина: `array_length(array_column, dimension)`
5. **Обновление:**
   - Полностью: `UPDATE table_name SET array_column = ARRAY['new1', 'new2']`

- Элемент: `array_column[index] = 'new_value'`

6. **Поиск в массиве:**
   - Наличие элемента: `'value' = ANY(array_column)`

Операторы работы с массивами

1. **Операторы сравнения:**
   - `=`, `>`, `<` — сравнивают массивы поэлементно, true, если значения и порядок совпадают.
   - Пример: `ARRAY[1, 2, 3] = ARRAY[1, 2, 3]` (true), `ARRAY[1, 2, 4] > ARRAY[1, 2, 3]` (true)
2. **Containment-операторы:**
   - `@>` — true, если левый массив содержит все элементы правого.
   - `<@` — true, если правый массив содержит все элементы левого.
   - Пример: `ARRAY[1, 2, 3] @> ARRAY[1, 2]` (true)
3. **Операторы пересечения:**
   - `&&` — true, если массивы имеют хотя бы один общий элемент.
   - Пример: `ARRAY[1, 2, 3] && ARRAY[2, 4]` (true)

Пользовательские типы

Домены

Домены — пользовательские типы данных с ограничениями, обеспечивающие валидацию данных на уровне типа.

- Создание:

```
CREATE DOMAIN domain_name AS data_type CONSTRAINTS;
```

Пример:

```
CREATE DOMAIN text_no_space_null AS TEXT NOT NULL CHECK (value !~ '\s');
```

Операции с доменами:

- Добавление ограничения:

```
ALTER DOMAIN domain_name ADD CONSTRAINT new_constraint [NOT VALID];
```

- Валидация существующих данных:

```
ALTER DOMAIN domain_name VALIDATE CONSTRAINT new_constraint;
```

Примечание: добавление ограничения с `NOT VALID` позволяет сначала внести его, не проверяя текущие данные, а затем провести валидацию после исправления данных.

Композитные типы данных в PostgreSQL

Составной тип — пользовательский тип данных, состоящий из нескольких полей с разными типами данных.

- **Использование:** часто применяется для возврата данных из функций, но не для хранения в таблицах.
- **Ограничения:** не поддерживает ограничения (constraints).

Создание типа:

```
CREATE TYPE type_name AS (
    field1 data_type,
    field2 data_type
);
```

Создание экземпляра:

- Используйте `ROW(value1, value2, ...)` для создания записи составного типа.

Доступ к полям:

```
SELECT (column_name).field1 FROM table_name;
```

Обновление полей:

```
UPDATE table_name SET column_name.field = new_value;
```

Удаление типа:

```
DROP TYPE IF EXISTS type_name [CASCADE];
```

Примечание: используйте таблицы с отдельными столбцами вместо композитных типов, если это возможно, для более удобного доступа и управления данными.

```
CREATE TYPE complex AS (
    r float8,
    i float8
);

CREATE TABLE math_calcs (
    math_id serial,
    val complex
);

INSERT INTO math_calcs(val) VALUES (ROW(3.0, 4.0)), (ROW(2.0, 1.0));

SELECT (val).r, (val).i FROM math_calcs; -- Доступ к отдельным полям

UPDATE math_calcs SET val.r = 3 WHERE math_id = 1; -- Обновление поля
```

Перечисления (ENUM) в PostgreSQL

Перечисление (ENUM) — пользовательский тип данных, заменяющий простейшие справочные таблицы.

- **Определение:**
  ```
  CREATE TYPE type_name AS ENUM ('value1', 'value2', ...);
  ```
- **Особенности:**
  - Значения регистрозависимы.
  - Нельзя сравнивать разные перечисления.
  - Ограничивает вставку в колонку значений, которые не определены в перечислении.

Использование в таблице:

```
CREATE TABLE table_name (
    column_name type_name
);
```

Добавление значений в ENUM:

```
ALTER TYPE type_name ADD VALUE 'new_value' [BEFORE 'existing_value' | AFTER 'existing_value'];
```

Проверка значений ENUM:

```
SELECT enum_range(NULL::type_name);
```

Вставка данных и ошибки:

- При вставке значения, отсутствующего в ENUM, будет ошибка.
- Вставка по индексу ENUM не поддерживается.

Примечание: перечисления упрощают хранение ограниченного набора значений, но требуют изменения структуры при добавлении новых значений.

```
CREATE TYPE chess_title AS ENUM
('Candidate Master', 'FIDE Master', 'International Master');

SELECT enum_range(null::chess_title);
--{"Candidate Master","FIDE Master","International Master"}

ALTER TYPE chess_title
ADD VALUE 'Grand Master' AFTER 'International Master'; -- OR BEFORE

SELECT enum_range(null::chess_title);
--{"Candidate Master","FIDE Master","International Master","Grand Master"}

CREATE TABLE chess_player (
    player_id serial PRIMARY KEY,
    first_name text,
    last_name text,
    title chess_title
);

INSERT INTO chess_player(first_name, last_name, title) VALUES ('Magnus',
'Carlsen', 'Grand Master');
```

---

8. Агрегация и группировка

COUNT

Подсчитывает количество строк, можно подсчитать уникальные значения.

```
SELECT COUNT([DISTINCT] столбец) FROM имя_таблицы;
```

MIN, MAX, AVG, SUM

Выполняет агрегатные функции для нахождения минимального, максимального, среднего значения или суммы.

```
SELECT MIN(столбец)/MAX(…)/AVG(…)/SUM(…) FROM имя_таблицы;
```

GROUP BY / HAVING

Группирует строки по указанным столбцам и фильтрует группы по условию с `HAVING`.

```
SELECT столбец1, SUM(столбец2)
FROM имя_таблицы
GROUP BY столбец1
HAVING условие
[ORDER BY SUM(столбец2) DESC];
```

FILTER

Условие для отдельного агрегата — несколько разных подсчётов в одном запросе.

```
SELECT category_id,
       COUNT(*)                                  AS total,
       COUNT(*) FILTER (WHERE unit_price > 50) AS expensive
FROM products
GROUP BY category_id;
```

GROUPING SETS, ROLLUP, CUBE

Группировка в SQL:

- **GROUP BY** используется для создания простых группировок данных.
- **GROUPING SETS, ROLLUP, CUBE** позволяют создавать подытоги и общие итоги в отчетах.

Описание и синтаксис

1. **GROUPING SETS:**
   - Позволяет объединять несколько наборов группировок.

   ```
   GROUP BY GROUPING SETS ((col_a), (col_a, col_b))
   ```

2. **ROLLUP:**

- Генерирует агрегированный набор для иерархии значений по указанным столбцам.

```
GROUP BY ROLLUP(col_a, col_b)
```

Эквивалент:

```
GROUP BY GROUPING SETS ((col_a), (col_a, col_b))
```

3. **CUBE:**

- Создает агрегированный набор для всех возможных комбинаций указанных столбцов.

```
GROUP BY CUBE(col_a, col_b)
```

Эквивалент всех комбинаций:

```
GROUP BY GROUPING SETS ((col_a), (col_b), (col_a, col_b), ())
```

Пример использования:

```
SELECT col_a, col_b, SUM(metric)
FROM table_name
GROUP BY ROLLUP(col_a, col_b);

SELECT col_a, col_b, SUM(metric)
FROM table_name
GROUP BY CUBE(col_a, col_b);
```

Примечание:

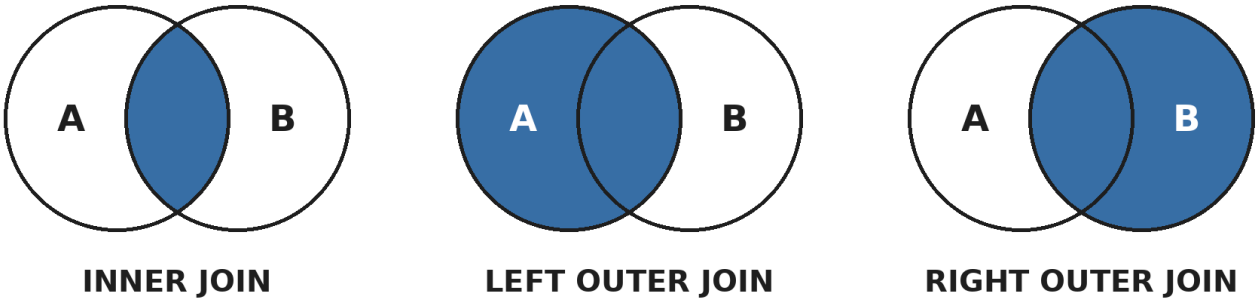
- Итоги, созданные с помощью `ROLLUP` и `CUBE`, могут содержать `NULL`, которые представляют агрегацию всех записей по определенным столбцам.

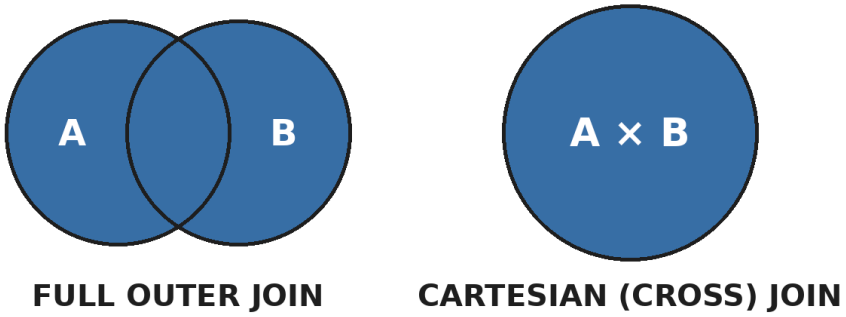
---

9. Объединение данных

JOIN

JOIN — это операция в SQL, которая позволяет объединить строки из двух или более таблиц на основе связанных между ними столбцов.





Виды JOIN

INNER JOIN

Возвращает только те строки, которые есть в обеих таблицах.

```
SELECT столбец1, столбец2
FROM таблица1
INNER JOIN таблица2 ON таблица1.столбец = таблица2.столбец;
```

LEFT JOIN (LEFT OUTER JOIN)

Возвращает все строки из левой таблицы, даже если нет соответствующих строк в правой таблице (пустые строки для правой таблицы будут NULL).

```
SELECT столбец1, столбец2
FROM таблица1
LEFT JOIN таблица2 ON таблица1.столбец = таблица2.столбец;
```

RIGHT JOIN (RIGHT OUTER JOIN)

Возвращает все строки из правой таблицы, даже если нет соответствующих строк в левой таблице (пустые строки для левой таблицы будут NULL).

```
SELECT столбец1, столбец2
FROM таблица1
RIGHT JOIN таблица2 ON таблица1.столбец = таблица2.столбец;
```

FULL OUTER JOIN

Возвращает все строки, когда есть соответствие в одной из таблиц. Если строка в одной из таблиц не имеет соответствия в другой, NULL заполняет отсутствующие значения.

```
SELECT столбец1, столбец2
FROM таблица1
FULL OUTER JOIN таблица2 ON таблица1.столбец = таблица2.столбец;
```

CROSS JOIN

Возвращает декартово произведение строк обеих таблиц. Каждая строка из первой таблицы соединяется с каждой строкой второй таблицы.

```
SELECT столбец1, столбец2
FROM таблица1
CROSS JOIN таблица2;
```

SELF JOIN

Соединяет таблицу саму с собой. Используется для сравнения строк в пределах одной таблицы.

```
SELECT e1.столбец, e2.столбец
FROM таблица e1
JOIN таблица e2 ON e1.ссылка = e2.id;
```

USING & NATURAL JOIN

USING

Упрощает соединение по одинаковым столбцам в обеих таблицах.

```
SELECT столбец1, столбец2
FROM таблица1
```

```
JOIN таблица2 USING(общий_столбец);
```

NATURAL JOIN

Соединяет таблицы автоматически по всем столбцам с одинаковыми именами. Рекомендуется избегать использования из-за его неявности и возможных ошибок.

```
SELECT *
FROM таблица1
NATURAL JOIN таблица2;
```

Псевдонимы (AS)

Используются для упрощения ссылок на таблицы и столбцы.

```
SELECT t1.столбец AS alias1, t2.столбец AS alias2
FROM таблица1 AS t1
JOIN таблица2 AS t2 ON t1.столбец = t2.столбец;
```

Где можно использовать псевдонимы:

- **SELECT:** для переименования столбцов или выражений.
- **ORDER BY:** псевдонимы можно использовать для сортировки.

Где нельзя использовать псевдонимы:

- **GROUP BY, HAVING, WHERE:** псевдонимы недоступны, так как эти блоки выполняются до их создания. Нужно использовать оригинальные названия столбцов или выражения.

Полный порядок — в разделе Порядок выполнения запроса.

UNION, INTERSECT, EXCEPT

Объединение, исключая дубликаты (`UNION`), без исключения дубликатов (`UNION ALL`), пересечение (`INTERSECT`) или исключение (`EXCEPT`) результатов двух запросов.

```
SELECT столбец1 FROM имя_таблицы1
[UNION|INTERSECT|EXCEPT]
SELECT столбец1 FROM имя_таблицы2;
```

Подзапросы

Подзапрос — это SQL-запрос, вложенный внутри другого запроса. Он используется для выполнения операций, результат которых необходим для основного запроса.

Случаи использования подзапросов:

- **Фильтрация данных:** использование результатов подзапроса в условиях `WHERE` или `HAVING`.
- **Выборка данных:** использование подзапроса в списке выбранных столбцов.
- **Создание виртуальных таблиц:** использование подзапроса в операторе `FROM`.

Подзапросы в разных частях запроса

Подзапросы в WHERE

Используется для фильтрации данных по результатам подзапроса.

```
SELECT […] FROM […]
WHERE […] IN (SELECT […] FROM […] WHERE […]);
```

Подзапросы в FROM (виртуальная таблица)

Подзапрос используется для создания виртуальной таблицы.

```
SELECT […]
FROM (SELECT […] FROM […] WHERE […]) AS подзапрос
```

Подзапросы в LIMIT

Используется для ограничения количества строк, возвращаемых основным запросом.

```
SELECT […] FROM […]
LIMIT (SELECT MIN(…) FROM […]);
```

Подзапросы в SELECT

Используется для вычисления значения в одном из столбцов.

```
SELECT (SELECT AVG(столбец2) FROM таблица2)[, столбец1,] AS среднее_значение
FROM таблица1;
```

WHERE [NOT] EXISTS

Проверяет, возвращает ли подзапрос хотя бы одну строку. Используется для проверки наличия связанных данных. Или в случае с `NOT` — что ни одна строка не возвращается.

```
SELECT *
FROM таблица1 t1
WHERE [NOT] EXISTS (
    SELECT 1
    FROM таблица2 t2
    WHERE t2.столбец = t1.столбец
);
```

ANY / ALL

- **ANY:** сравнивает значение с любым из значений, возвращаемых подзапросом. Если хотя бы одно значение удовлетворяет условию, результат будет истинным.
- **ALL:** условие истинно, только если оно выполняется для всех значений, возвращаемых подзапросом.

```
SELECT *
FROM таблица
WHERE столбец > ANY | ALL (
    SELECT значение
    FROM таблица2
    WHERE […]
);
```

Коррелированные подзапросы

Это подзапросы, которые ссылаются на столбцы из внешнего запроса. Они выполняются для каждой строки внешнего запроса.

```
SELECT столбец1
FROM таблица1 t1
WHERE (SELECT столбец2 FROM таблица2 t2 WHERE t2.ссылка = t1.ссылка) =
значение;
```

Общие табличные выражения (CTE) в SQL

CTE (Common Table Expressions) — временные таблицы, которые используются для повышения читабельности и чистоты кода в сложных запросах. Они создают временные подзапросы, которые можно переиспользовать в рамках одного запроса.

Синтаксис CTE:

```
WITH cte_name AS (
    SELECT ...
)
SELECT ... FROM cte_name;
```

- С PostgreSQL 12: если CTE используется **один раз** (и без побочных эффектов), планировщик встраивает его в основной запрос как подзапрос; если **несколько раз** — вычисляет один раз и кеширует. Поведение можно задать явно:

  ```
  WITH cte_name AS MATERIALIZED (...)      -- вычислить один раз (старое
  поведение, до PG 12)
  WITH cte_name AS NOT MATERIALIZED (...)  -- встроить в запрос
  ```

- Можно использовать **INSERT, UPDATE, DELETE** внутри CTE.
- **CTE** помогает разбирать сложные запросы на части для упрощения понимания.

Рекурсивные CTE

Рекурсивные CTE позволяют строить запросы, где требуется повторяющаяся логика, например, для построения иерархических структур (деревьев).

Синтаксис рекурсивного CTE:

```
WITH RECURSIVE cte_name AS (
    SELECT ... -- начальная часть
    UNION ALL
    SELECT ... -- рекурсивная часть, использующая ссылку на CTE
)
SELECT * FROM cte_name;
```

Применение:

- Построение иерархических структур (например, цепочек сотрудников и их менеджеров).
- Решение задач, требующих многократных итераций в рамках одного запроса.

---

10. Оконные функции

Оконные функции выполняют вычисления по набору строк (окну), связанных с текущей строкой, и возвращают результат для каждой строки без группировки данных.

Основные элементы оконных функций

1. **Партиция** — это деление набора данных на подмножества (партиции), где строки сгруппированы по одному или нескольким столбцам. Оконные функции применяются отдельно для каждой партиции.
2. **Окно** — определяет, какие строки в пределах каждой партиции будут участвовать в вычислении для каждой строки. Окно может изменяться от строки к строке (особенно в случаях с границами окна, когда берется не вся партиция, а ее часть).

   Если нет ни `ORDER BY`, ни `ROWS` / `RANGE` — окно совпадает с партицией (подробнее — «Рамка по умолчанию» ниже).

```
<оконная_функция>(<поле>) OVER (
    [PARTITION BY <столбец_для_разделения>]
    [ORDER BY <столбец_для_сортировки>]
    [ROWS|RANGE BETWEEN <границы_окна>]
)
```

- **OVER ()** без параметров — окно = вся выборка.
- **PARTITION BY** — делит выборку на партиции по значениям столбцов.
- **ORDER BY** — порядок строк внутри партиции (обязателен для ранжирования и `LAG/LEAD`).

Основные оконные функции

1. **Агрегатные оконные функции** — вычисляют агрегатные значения для строк в пределах окна:
   - `SUM()` — подсчитывает сумму значений.
   - `COUNT()` — считает количество строк.
   - `AVG()` — находит среднее значение.
   - `MAX()` — возвращает наибольшее значение.
   - `MIN()` — возвращает наименьшее значение.
2. **Ранжирующие оконные функции** — присваивают каждой строке её «ранг» или позицию в пределах окна:
   - `ROW_NUMBER()` — нумерация строк.
   - `RANK()` — ранжирует строки с пропусками рангов при одинаковых значениях.
   - `DENSE_RANK()` — ранжирует строки без пропусков.
3. **Функции смещения** — обращаются к предыдущим или следующим строкам в пределах окна:
   - `LAG()` — возвращает значение из предыдущей строки.
   - `LEAD()` — возвращает значение из следующей строки.
   - `FIRST_VALUE()` — возвращает первое значение в окне.

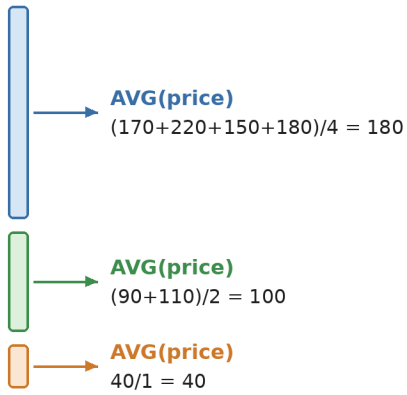
- `LAST_VALUE()` — возвращает последнее значение в окне.

```
SELECT
    id,
    price,
    LAG(price, 1) OVER (ORDER BY id) AS previous_price -- часть строки на
скане закрыта рисунком
FROM Rooms;
```

Оконная функция без ROWS/RANGE

```
AVG(price) OVER (PARTITION BY home_type)
```

home_type	price	avg_price
Entire home/apt	170	**180**
Entire home/apt	220	**180**
Entire home/apt	150	**180**
Entire home/apt	180	**180**
Private room	90	**100**
Private room	110	**100**
Shared room	40	**40**

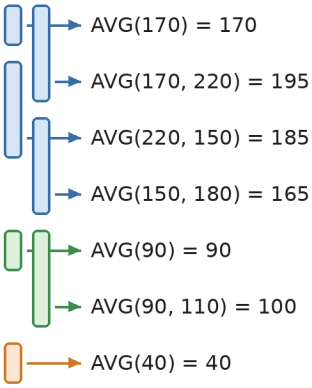


Окно = вся партиция: каждая строка партиции получает одно и то же среднее.

Оконная функция с ROWS/RANGE

```
AVG(price) OVER (PARTITION BY home_type ORDER BY id
                 ROWS BETWEEN 1 PRECEDING AND CURRENT ROW)
```

home_type	price	avg_price
Entire home/apt	170	**170**
Entire home/apt	220	**195**
Entire home/apt	150	**185**
Entire home/apt	180	**165**
Private room	90	**90**
Private room	110	**100**
Shared room	40	**40**



Окно = предыдущая строка + текущая (в пределах партиции): своё окно у каждой строки.

ROW_NUMBER vs RANK vs DENSE_RANK

```sql
SELECT price,
       ROW_NUMBER() OVER (ORDER BY price DESC) AS row_number,
       RANK()       OVER (ORDER BY price DESC) AS rank,
```

```
       DENSE_RANK() OVER (ORDER BY price DESC) AS dense_rank
FROM Rooms;
```

price	row_number	rank	dense_rank
220	1	1	1
180	2	2	2
180	3	2	2
170	4	4	3

Типичная задача — «топ-N в каждой группе»: `ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)` в подзапросе/CTE + `WHERE rn <= N`.

Окна и диапазоны

Окно можно ограничить с помощью ключевых слов `ROWS` и `RANGE`, определяя количество строк или диапазон значений, которые будут использоваться для каждой строки в окне.

ROWS — основан на физическом количестве строк:

- При использовании `ROWS` окно ограничивается конкретным числом строк относительно текущей. Например, `1 PRECEDING` означает одну строку до текущей.
- **Точная граница:** `ROWS` определяет точное количество строк, которые войдут в окно, что делает его предсказуемым.

RANGE — основан на значениях:

- В отличие от `ROWS`, `RANGE` определяет границы окна на основе значений столбцов. Это значит, что окно может варьироваться в зависимости от значений в наборе данных.
- **Динамичность границ:** границы окна могут изменяться в зависимости от данных, что делает их гибкими, но менее предсказуемыми.

⚠ Рамка по умолчанию

- `OVER (PARTITION BY ...)` без `ORDER BY` — окно = вся партиция.
- `OVER (... ORDER BY ...)` без `ROWS/RANGE` — окно = `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, т.е. от начала партиции до текущей строки (и всех строк с тем же значением `ORDER BY`).

Отсюда частая ошибка: `LAST_VALUE(x) OVER (ORDER BY id)` возвращает значение **текущей** строки. Для последнего значения партиции рамку задают явно:

```
LAST_VALUE(price) OVER (ORDER BY id ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)
```

Границы окон

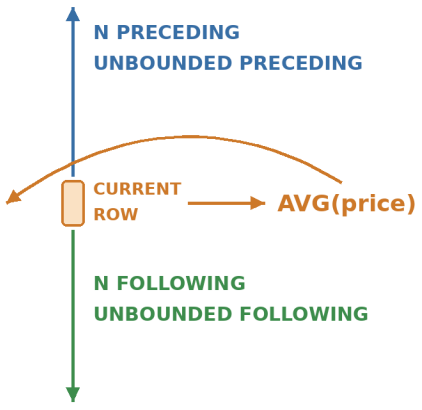
Границы окна определяются относительно текущей строки. Возможные границы окна:

- **UNBOUNDED PRECEDING** — все строки до текущей строки.
- **N PRECEDING** — N строк до текущей строки.
- **CURRENT ROW** — текущая строка.
- **N FOLLOWING** — N строк после текущей строки.
- **UNBOUNDED FOLLOWING** — все строки после текущей строки.

```
SELECT
    id,
    price,
    SUM(price) OVER (ORDER BY id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
AS rolling_sum
FROM Rooms;
```

Границы окна относительно текущей строки

id	home_type	price	avg_price
1	Entire home/apt	170	180
2	Entire home/apt	220	180
3	**Entire home/apt**	**150**	**180**
4	Entire home/apt	90	100
5	Entire home/apt	110	100
6	Private room	40	40



Строки выше текущей — PRECEDING, ниже — FOLLOWING; окно задаётся как BETWEEN <начало> AND <конец>.

Порядок выполнения запроса

`FROM / JOIN → WHERE → GROUP BY → HAVING → SELECT (здесь же оконные функции) → DISTINCT → ORDER BY → LIMIT`

- Псевдонимы из `SELECT` не видны в `WHERE` и `HAVING` — они вычисляются раньше. В `ORDER BY` видны; в `GROUP BY` PostgreSQL их тоже допускает (расширение стандарта, в других СУБД может не работать).
- Оконные функции считаются после `WHERE/GROUP BY/HAVING`, поэтому фильтровать по их результату можно только во внешнем запросе (подзапрос/CTE).

---

11. Объекты БД

Представления (View)

VIEW (представление) — это виртуальная таблица, основанная на результате SQL-запроса. Представление не хранит данные самостоятельно, а предоставляет определенный способ просмотра данных из одной или нескольких таблиц.

Зачем: спрятать сложный запрос за именем таблицы; дать доступ только к части данных (права на view, а не на таблицы).

Виды представлений:

1. **Временные представления:** используются в течение одной сессии.
2. **Материализованные представления:** кэшируют результат запроса и позволяют улучшить производительность при частом доступе.
3. **Обновляемые представления:** позволяют выполнять `INSERT`, `UPDATE` и `DELETE`, если они не содержат сложных операций.
4. **Рекурсивные представления:** используются для обработки рекурсивных данных.

Создание и использование представлений

```
CREATE VIEW view_name AS
SELECT select_statement
```

Ограничение через WITH CHECK OPTION

`WITH LOCAL CHECK OPTION` гарантирует, что данные, вставленные или обновлённые через представление, будут соответствовать условиям фильтрации, заданным в представлении.

```
CREATE VIEW heavy_orders AS
SELECT * FROM orders WHERE freight > 100
WITH LOCAL CHECK OPTION;
```

Изменение представлений

- Можно только добавить новые столбцы, нельзя удалить существующий, нельзя поменять имена столбцов, нельзя поменять даже порядок следования столбцов:

```
CREATE OR REPLACE VIEW view_name AS
SELECT select_statement
```

- Можно переименовывать представления:

```
ALTER VIEW old_view RENAME TO new_view_name
```

- Можно удалять представления:

```
DROP VIEW [IF EXISTS] view_name
```

Модифицировать данные через представление можно, если оно ссылается на одну таблицу и не содержит таких операций, как `GROUP BY`, `HAVING`, `UNION` и т.д.

Функции в SQL и PL/pgSQL

Зачем нужны функции в SQL?

Логика хранится и выполняется на сервере БД: меньше сетевых round-trip'ов, одна реализация для всех клиентов. Внутри — любые `SELECT/INSERT/UPDATE/DELETE`, но нельзя `COMMIT`, `SAVEPOINT` и утилитарные команды (`VACUUM`). Виды:

1. **SQL-функции** — выполняют SQL-запросы.
2. **Процедурные (PL/pgSQL)** — поддерживают более сложную логику с циклами и условиями.
3. **Серверные функции на C** — редко используются.

Синтаксис создания функции:

```
CREATE FUNCTION func_name([arg1, arg2...]) RETURNS data_type AS $$
-- логика функции
$$ LANGUAGE lang;
```

IN, OUT, DEFAULT

- **IN** — входящие аргументы
- **OUT** — исходящие аргументы
- **INOUT** — и входящий, и исходящий аргумент (не рекомендуется)
- **VARIADIC** — массив входящих параметров

- **DEFAULT** value

IN (входящий аргумент):

```
CREATE OR REPLACE FUNCTION get_product_price_by_name([IN] prod_name varchar)
RETURNS real AS $$
    SELECT unit_price
    FROM products
    WHERE product_name = prod_name;
$$ LANGUAGE SQL;

-- Пример вызова:
SELECT get_product_price_by_name('Chocolade'); -- Вернёт цену продукта
```

OUT (исходящий аргумент):

```
CREATE OR REPLACE FUNCTION get_price_boundaries(OUT max_price real, OUT
min_price real) AS $$
    SELECT MAX(unit_price), MIN(unit_price)
    FROM products;
$$ LANGUAGE SQL;

-- Пример вызова:
SELECT * FROM get_price_boundaries(); -- Вернёт максимальную и минимальную
цену
```

INOUT (и входящий, и исходящий аргумент, в большинстве не рекоменд.):

```
CREATE OR REPLACE FUNCTION adjust_price(INOUT price real) AS $$
BEGIN
    IF price < 10 THEN
        price := price + 5;
    ELSE
        price := price - 2;
    END IF;
END;
$$ LANGUAGE plpgsql;

-- Пример вызова:
SELECT adjust_price(15); -- Вернёт 13 (уменьшенная цена)
```

VARIADIC (массив входящих параметров):

```
CREATE OR REPLACE FUNCTION sum_prices(VARIADIC prices real[]) RETURNS real
AS $$
    SELECT SUM(p) FROM unnest(prices) AS p;
$$ LANGUAGE SQL;

-- Пример вызова:
SELECT sum_prices(10, 20, 30); -- Вернёт сумму: 60
```

DEFAULT (значение по умолчанию):

```
CREATE OR REPLACE FUNCTION get_discounted_price(price real, discount real
DEFAULT 0.1) RETURNS real AS $$
    SELECT price * (1 - discount);
$$ LANGUAGE SQL;

-- Пример вызова:
SELECT get_discounted_price(100);      -- Вернёт 90 (цена с учётом скидки
10%)
SELECT get_discounted_price(100, 0.2); -- Вернёт 80 (цена с учётом скидки
20%)
```

Возврат наборов данных

- `RETURNS SETOF data_type` — возврат n-значений типа `data_type`
- `RETURNS SETOF table` — если нужно вернуть все столбцы из таблицы или пользовательского типа
- `RETURNS SETOF record` (не рекомендуется без необходимости, ибо как с анонимным типом работаем) (только когда типы колонок в результирующем наборе заранее неизвестны)
- `RETURNS TABLE (column_name data_type, ...)` — то же, что и `SETOF table`, но имеем возможность явно указать возвращаемые столбцы
- Возврат через OUT-параметры

RETURNS SETOF data_type — возвращает набор значений определенного типа:

```
CREATE FUNCTION get_average_prices() RETURNS SETOF real AS $$
    SELECT AVG(unit_price) FROM products GROUP BY category_id;
$$ LANGUAGE SQL;

SELECT * FROM get_average_prices();
```

RETURNS SETOF table — возвращает все столбцы из таблицы:

```
CREATE FUNCTION get_products() RETURNS SETOF products AS $$
    SELECT * FROM products;
$$ LANGUAGE SQL;

SELECT * FROM get_products();
```

RETURNS SETOF record — возвращает набор строк без явного определения типов (не рекомендуется):

```
CREATE FUNCTION get_product_details() RETURNS SETOF record AS $$
    SELECT product_name, unit_price FROM products;
$$ LANGUAGE SQL;

SELECT * FROM get_product_details() AS (product_name text, unit_price real);
```

RETURNS TABLE (column_name data_type, ...) — явное указание возвращаемых столбцов и их типов:

```
CREATE FUNCTION get_products_by_category() RETURNS TABLE (category_name
text, avg_price real) AS $$
    SELECT category_name, AVG(unit_price) FROM products JOIN categories
    USING(category_id) GROUP BY category_name;
$$ LANGUAGE SQL;

SELECT * FROM get_products_by_category();
```

Возврат через OUT-параметры — возвращает набор данных через выходные параметры.

Введение в plpgsql

1. **DECLARE:** используется для объявления переменных внутри функции.
2. **BEGIN / END:** задают блок кода, в котором происходит выполнение логики функции.
3. **Логика и управление:** функции PL/pgSQL поддерживают условные конструкции (`IF` , `CASE`), циклы (`FOR` , `WHILE`) и блоки обработки исключений.
4. **RETURN:** для возврата значения из функции. В PL/pgSQL можно использовать `RETURN QUERY` , чтобы вернуть результаты сложного запроса.

```
CREATE FUNCTION get_discount(price real) RETURNS real AS $$
DECLARE
    discount real;
BEGIN
```

```
    IF price > 100 THEN
        discount := price * 0.10; -- скидка 10%
    ELSE
        discount := price * 0.05; -- скидка 5%
    END IF;

    RETURN discount;
END;
$$ LANGUAGE plpgsql;

SELECT get_discount(150); -- 15.0
```

Оператор `:=` выполняет присвоение значения переменной, позволяя вычислять или сохранять данные для дальнейшего использования в теле функции.

IF ELSE

Блок `IF` используется в PL/pgSQL для выполнения условной логики. В зависимости от условия выполняется одна из ветвей кода.

```
IF expression THEN
  logic
ELSEIF expression THEN
  logic
ELSE
  logic
END IF;
```

Циклы

Циклы в **PL/pgSQL** позволяют выполнять повторяющиеся операции в пределах функций. Основные типы циклов:

1. **WHILE:** выполняет блок логики, пока условие истинно.

   ```
   WHILE condition LOOP
     -- logic
   END LOOP;
   ```

2. **LOOP + EXIT:** выполняет логику до тех пор, пока не выполнится условие выхода.

   ```
   LOOP
     EXIT WHEN condition;
     -- logic
   END LOOP;
   ```

3. **FOR:** итерирует заданное количество раз, можно задать шаг через `BY`.

```
FOR counter IN a..b LOOP
  -- logic
END LOOP;
```

RETURN NEXT

RETURN NEXT — оператор в PL/pgSQL, который позволяет добавлять записи в результирующий набор построчно. Этот подход полезен для накопления данных, особенно в ситуациях, где требуются циклы или условия для обработки набора записей.

Применение и ограничения:

- `RETURN NEXT` используется, когда функция возвращает `SETOF` (набор данных) и позволяет добавлять данные в результат итеративно.
- Построчная обработка данных с помощью `RETURN NEXT` может быть менее производительна, чем декларативный SQL, особенно при работе с большим объемом данных.
- По возможности старайтесь избегать циклов и использовать чистый SQL — он проще для восприятия и оптимизации.

```
CREATE OR REPLACE FUNCTION after_christmas_sale() RETURNS SETOF products AS
$$
DECLARE
    product record;
BEGIN
    FOR product IN SELECT * FROM products
    LOOP
        IF product.category_id IN (1, 4, 8) THEN
            product.unit_price := product.unit_price * 0.8;
        ELSEIF product.category_id IN (2, 3, 7) THEN
            product.unit_price := product.unit_price * 0.75;
        ELSE
            product.unit_price := product.unit_price * 1.1;
        END IF;
        RETURN NEXT product;
    END LOOP;
END;
$$ LANGUAGE plpgsql;

SELECT * FROM after_christmas_sale();
```

Обработка ошибок

В PL/pgSQL ошибки обрабатываются с помощью операторов `RAISE` и блоков `EXCEPTION`. Структура включает указание уровня ошибки, сообщения и дополнительных параметров, таких как `HINT` и `ERRCODE`.

Структура RAISE

- **RAISE** используется для генерации ошибок и сообщений:

```
RAISE [level] 'message', param
    USING HINT = 'Hint message', ERRCODE = 'Custom code';
```

  - **level** — уровень ошибки:
    - `DEBUG` — отладка
    - `LOG` — запись в лог
    - `INFO` — информация
    - `NOTICE` — уведомление
    - `WARNING` — предупреждение
    - `EXCEPTION` — ошибка, прерывающая выполнение
  - **USING HINT** — подсказка для исправления ошибки.
  - **ERRCODE** — код ошибки (может быть стандартным или кастомным).

Пример RAISE с параметрами:

```
RAISE EXCEPTION 'Invalid input value: %', input_value
    USING HINT = 'Value must be between 1 and 10', ERRCODE = 'P0001';
```

Блок обработки исключений (EXCEPTION)

Используйте блок `EXCEPTION` для перехвата и обработки ошибок:

```
BEGIN
    -- основной код
EXCEPTION
    WHEN condition THEN
        -- обработка конкретной ошибки
        RAISE INFO 'Handled error: %', SQLERRM;
    WHEN OTHERS THEN
        -- обработка всех других ошибок
        RAISE INFO 'Unhandled error: %', SQLERRM;
END;
```

- **condition** — условие для перехвата конкретных ошибок (`SQLSTATE` или код ошибки).
- **WHEN OTHERS** — перехват всех остальных ошибок.

Массивы в функциях: VARIADIC и FOREACH

- **VARIADIC:** используется для передачи переменного количества параметров одного типа.

```
CREATE OR REPLACE FUNCTION example_func(VARIADIC arg_name int[])
RETURNS void AS $$
BEGIN
  -- logic
END;
$$ LANGUAGE plpgsql;
```

- **FOREACH:** позволяет итерировать по элементам массива в PL/pgSQL.

```
FOREACH element_var IN ARRAY array_name
LOOP
  -- logic
END LOOP;
```

Триггеры в PostgreSQL

Основы триггеров

- **Триггер** — это объект, выполняющий действия при наступлении определённых событий в таблице.
- Используется для:
  - Аудита изменений.
  - Дополнительных операций при вставке, обновлении или удалении данных.
  - Проверок целостности данных.
- Срабатывает построчно или единоразово для всех изменений.

Создание и использование триггеров

- **Синтаксис создания триггера:**

```
CREATE TRIGGER trigger_name
{ BEFORE | AFTER } { INSERT | UPDATE | DELETE }
ON table_name
FOR EACH ROW EXECUTE FUNCTION function_name();  -- PG 11+; EXECUTE
PROCEDURE — старый синоним
```

Функции для триггеров

- **Функция, выполняемая триггером:**

```
CREATE FUNCTION func_name() RETURNS trigger AS $$
BEGIN
  -- Логика триггера
END;
$$ LANGUAGE plpgsql;
```

- Функция должна возвращать `NULL` или запись, соответствующую структуре таблицы.

Аргументы функции триггера

- **OLD** — аргумент, содержащий данные существующей строки до изменений. Используется при `UPDATE` и `DELETE`.
- **NEW** — аргумент, содержащий новые данные при вставке или обновлении строки. Используется при `INSERT` и `UPDATE`.

Особенности работы триггеров

- **BEFORE-триггер:**
  - Возврат `NULL` отменяет выполнение операции.
  - Модификация строки через `NEW` позволяет изменить данные до выполнения операции.
  - Возвращает `OLD` при `DELETE`.
- **AFTER-триггер:**
  - Выполняет действия после завершения операции.
  - Возвращаемое значение игнорируется.
  - Для отмены операции используется `RAISE EXCEPTION`.

Управление триггерами

- **Удаление триггера:**

```
DROP TRIGGER IF EXISTS trigger_name ON table_name;
```

- **Переименование триггера:**

```
ALTER TRIGGER trigger_name ON table_name RENAME TO new_trigger_name;
```

- **Отключение триггера:**

```
ALTER TABLE table_name DISABLE TRIGGER trigger_name;
ALTER TABLE table_name DISABLE TRIGGER ALL; -- отключить все триггеры
```

Триггеры на утверждения

- Применяются для обработки событий целиком.
- Используется с конструкцией `REFERENCING NEW TABLE` или `OLD TABLE` для доступа ко всем новым или старым строкам.

Пример создания:

```
CREATE TRIGGER trigger_name
AFTER INSERT ON table_name
REFERENCING NEW TABLE AS new_table_name
FOR EACH STATEMENT
EXECUTE FUNCTION function_name();
```

---

12. Индексы и производительность

Индекс — это структура данных, предназначенная для ускорения поиска информации в таблице за счёт дополнительного объёма записей и занимаемого пространства на диске.

Устройство базы данных PostgreSQL

1. **Кластер базы данных:** несколько баз данных, управляемых одной серверной инстанцией, хранятся в директории данных (`PGDATA`).
2. **Файлы кластера:** каждая база данных имеет папку в `PGDATA/base`, где для каждой таблицы и индекса создаётся отдельный файл.
3. **Heap File:** основной файл данных для таблицы, представляющий собой массив страниц фиксированного размера (8 КБ), содержащих строки.

Структура таблиц PostgreSQL

- **Страницы таблицы:**
  - Разделены на 8 КБ блоки, включающие заголовок страницы и ссылки на строки (CTID).
  - Если значение столбца превышает 8 КБ, используется TOAST (The Oversized-Attribute Storage Technique): данные делятся на куски по 2 КБ и хранятся в связанной таблице TOAST.
- **Служебные файлы:**
  - **FSM (Free Space Map):** файл с информацией о свободном пространстве для ускорения вставок.
  - **VM (Visibility Map):** хранит флаги видимости страниц для оптимизации доступа к данным.

Поддержка и очистка данных

- **VACUUM:** удаляет «устаревшие» версии строк, оставляя пространство для новых данных.
- **VACUUM FULL:** полностью перестраивает таблицу, освобождая память (используется редко из-за ресурсоёмкости).
- **VACUUM ANALYZE:** обновляет статистику таблицы для оптимизации запросов.
- **Автоматический вакуум:** демон autovacuum запускает `VACUUM` автоматически для поддержания базы данных в хорошем состоянии.

Индексы и методы сканирования в PostgreSQL

Основные методы сканирования таблиц

Метод	Когда выбирается	Как работает
Index Scan	мало строк (высокая селективность)	индекс → для каждой записи прыжок в таблицу; на HDD случайный доступ дорогой
Index Only Scan	все нужные столбцы есть в индексе	читает только индекс; страницы проверяет по Visibility Map, поэтому нужен свежий `VACUUM`
Bitmap Scan	строк средне / много	сначала собирает битовую карту страниц по индексу, потом читает каждую страницу один раз
Seq Scan	большая часть таблицы	полный последовательный обход — быстрее случайного доступа

Виды индексов в PostgreSQL

1. B-tree (Balanced Tree)

- **Создается по умолчанию:** `CREATE INDEX index_name ON table_name (column_name)`
- **Поддерживаемые операции:** `<`, `>`, `<=`, `>=`, `=`, а также `LIKE 'abc%'` (но не `'%abc'`).
  - `LIKE 'abc%'` использует индекс, только если БД в локали `C` или индекс создан с `text_pattern_ops`:
    `CREATE INDEX ON table_name (column_name text_pattern_ops);`
- **Индексация NULL:** поддерживается.
- **Эффективность:** сложность поиска `O(logN)`, подходящий индекс для большинства случаев.

2. Hash

- **Создание:** `CREATE INDEX index_name ON table_name USING HASH (column_name)`
- **Поддерживаемые операции:** только `=`.
- **Особенности:** с PostgreSQL 10 пишется в WAL и реплицируется, так что безопасен при сбоях (до 10-й версии — нет).
- **Когда:** поиск `O(1)`, но только `=`; имеет смысл для длинных ключей, где он компактнее B-tree.

3. Специализированные индексы

- **GiST (Generalized Search Tree):** для геометрических и текстовых данных, подходит для полнотекстового поиска.
- **GIN (Generalized Inverted Index):** подходит для индексации массивов, JSONB, диапазонов, полнотекстового поиска. Более детализирован в запросах, занимает больше места, но быстрее на специфических данных.
- **SP-GiST (Space-Partitioned Generalized Search Tree):** для данных с естественным порядком, например, телефонные номера, где данные не сбалансированы.
- **BRIN (Block Range INdex):** оптимален для больших данных с упорядоченностью, таких как временные метки или почтовые индексы.

EXPLAIN и ANALYZE

EXPLAIN — команда для диагностики производительности. Показывает план выполнения запроса, что позволяет понять, где возникают узкие места.

- `EXPLAIN query` — отображает план запроса без выполнения.
- `EXPLAIN ANALYZE query` — выполняет запрос и отображает как план, так и реальные затраты на выполнение.

ANALYZE — собирает статистику по данным таблицы, улучшая планы выполнения запросов за счёт более точных данных о распределении значений.

- **Команда:** `ANALYZE [table_name [(column1, column2...)]]`
- **Автоматизация:** autovacuum (включен по умолчанию) также выполняет `ANALYZE`.

Важно: в чужой БД autovacuum может быть отключен или настроен иначе.

Построение индексов — базовый и составной

1. Создание простого индекса

```
CREATE INDEX idx_single_column ON table_name(column_name);
```

```
EXPLAIN SELECT * FROM table_name WHERE column_name = value; -- Используется
Index Scan
```

2. Создание составного индекса (по двум и более колонкам)

```
CREATE INDEX idx_composite ON table_name(column1, column2);
```

- Работает для фильтра по обеим колонкам или только по **первой** (`column1`). Порядок столбцов важен.
- Фильтр только по `column2` индекс почти не ускорит — нужен отдельный индекс на `column2` .

```
EXPLAIN SELECT * FROM table_name WHERE column1 = value1 AND column2 =
value2;
-- Index Scan
```

Индексы по выражениям и их особенности

Обычный индекс по `column_name` не используется для условия `WHERE LOWER(column_name) = ...` . Индекс строится по самому выражению:

```
CREATE INDEX idx_column_lower ON table_name(LOWER(column_name));
-- работает для WHERE LOWER(column_name) = 'abc'
```

Планировщик: без индекса — `Seq Scan` / `Parallel Seq Scan` , с индексом — обычно `Bitmap Index Scan` + `Bitmap Heap Scan` . Если условие отбирает большую часть таблицы, планировщик всё равно выберет `Seq Scan` — индекс выгоден только на небольших выборках.

Индексы для полнотекстового поиска

Для поиска подстрок используется расширение `pg_trgm` :

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE INDEX trgm_idx_column ON table_name USING gin (column_name
gin_trgm_ops);
```

Ускоряет `LIKE '%substring%'` и `ILIKE` ; если совпадений много — планировщик всё равно может выбрать `Seq Scan` .

---

13. Транзакции

Транзакция — это логическая группа операций, которая выполняется целиком или не выполняется вовсе. Примером служит банковская операция, где перевод средств включает несколько этапов, и если один из них не удается, вся транзакция откатывается.

Принципы ACID:

1. **Атомарность (Atomicity)** — транзакция либо полностью выполняется, либо не выполняется совсем. При ошибке все изменения откатываются.
2. **Согласованность (Consistency)** — система остается в согласованном состоянии до и после выполнения транзакции, соблюдая все ограничения и правила.
3. **Изолированность (Isolation)** — параллельные транзакции изолированы друг от друга, чтобы избежать несогласованности данных.
4. **Долговечность (Durability)** — после завершения транзакции её результаты сохраняются даже в случае сбоя.

Основные команды управления транзакциями (TCL):

- `BEGIN` или `START TRANSACTION` — начало транзакции.
- `COMMIT` — фиксация изменений.
- `ROLLBACK` — откат транзакции при ошибке.
- `SAVEPOINT savepoint_name` — создание точки сохранения внутри транзакции.
- `ROLLBACK TO savepoint_name` — откат к указанной точке сохранения.

Пример транзакции:

```
BEGIN;
  UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
  UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
COMMIT;
```

Особенности транзакций в PostgreSQL:

- Транзакциями нельзя управлять внутри **функций** — функция выполняется в транзакции вызывающего кода. Но с PostgreSQL 11 есть **процедуры**, в которых `COMMIT` / `ROLLBACK` разрешены:

```
CREATE PROCEDURE archive_orders() AS $$
BEGIN
    INSERT INTO orders_archive SELECT * FROM orders WHERE order_date <
'2020-01-01';
    COMMIT;
```

```
    DELETE FROM orders WHERE order_date < '2020-01-01';
    COMMIT;
END;
$$ LANGUAGE plpgsql;

CALL archive_orders();  -- процедура вызывается через CALL, не SELECT
```

- Для прерывания транзакции из функции можно использовать `RAISE EXCEPTION`.
- Частичный откат можно эмулировать с помощью комбинации `BEGIN` и `EXCEPTION WHEN`.

Для сравнения: в MS SQL транзакциями можно управлять прямо в хранимом коде; в PostgreSQL и MySQL — из внешнего кода (в PG ещё из процедур).

Уровни изоляции

Уровни изоляции в транзакциях определяют, как параллельно выполняемые транзакции взаимодействуют с данными, чтобы избежать конфликтов.

Проблемы параллельности:

- **«Грязное» чтение:** чтение данных, которые были изменены, но не зафиксированы.
- **Неповторяемое чтение:** данные, прочитанные повторно в одной транзакции, могут отличаться из-за изменений в других транзакциях.
- **Фантомное чтение:** повторное чтение возвращает другой набор строк из-за вставки новых данных другими транзакциями.
- **Аномалия сериализации:** несогласованные результаты при выполнении параллельных транзакций, которые могли бы отличаться при последовательном выполнении.

Сравнение уровней

Уровень изоляции	«Грязное» чтение	Неповторяемое чтение	Фантомное чтение	Аномалия сериализации
Read uncommitted (Чтение незафиксированных данных)	Допускается, но не в PG	Возможно	Возможно	Возможно
Read committed (Чтение зафиксированных данных, default)	Невозможно	Возможно	Возможно	Возможно

Уровень изоляции	«Грязное» чтение	Неповторяемое чтение	Фантомное чтение	Аномалия сериализации
Repeatable read (Повторяемое чтение)	Невозможно	Невозможно	Допускается, но не в PG	Возможно
Serializable (Сериализуемость)	Невозможно	Невозможно	Невозможно	Невозможно

В PostgreSQL `READ UNCOMMITTED` работает как `READ COMMITTED`. Чем строже уровень — тем больше проверок, ниже параллелизм и чаще откаты, поэтому по умолчанию `READ COMMITTED`.

MVCC (Multiversion Concurrency Control):

- Использует версионирование данных с таймштампами.
- Обеспечивает изоляцию за счёт снимков данных, снижая необходимость блокировок.

Синтаксис:

- Для начала транзакции с уровнем изоляции:
  ```
  BEGIN ISOLATION LEVEL level;
  ```
- Для установки уровня внутри транзакции:
  ```
  SET TRANSACTION ISOLATION LEVEL level;
  ```

Доступные уровни:

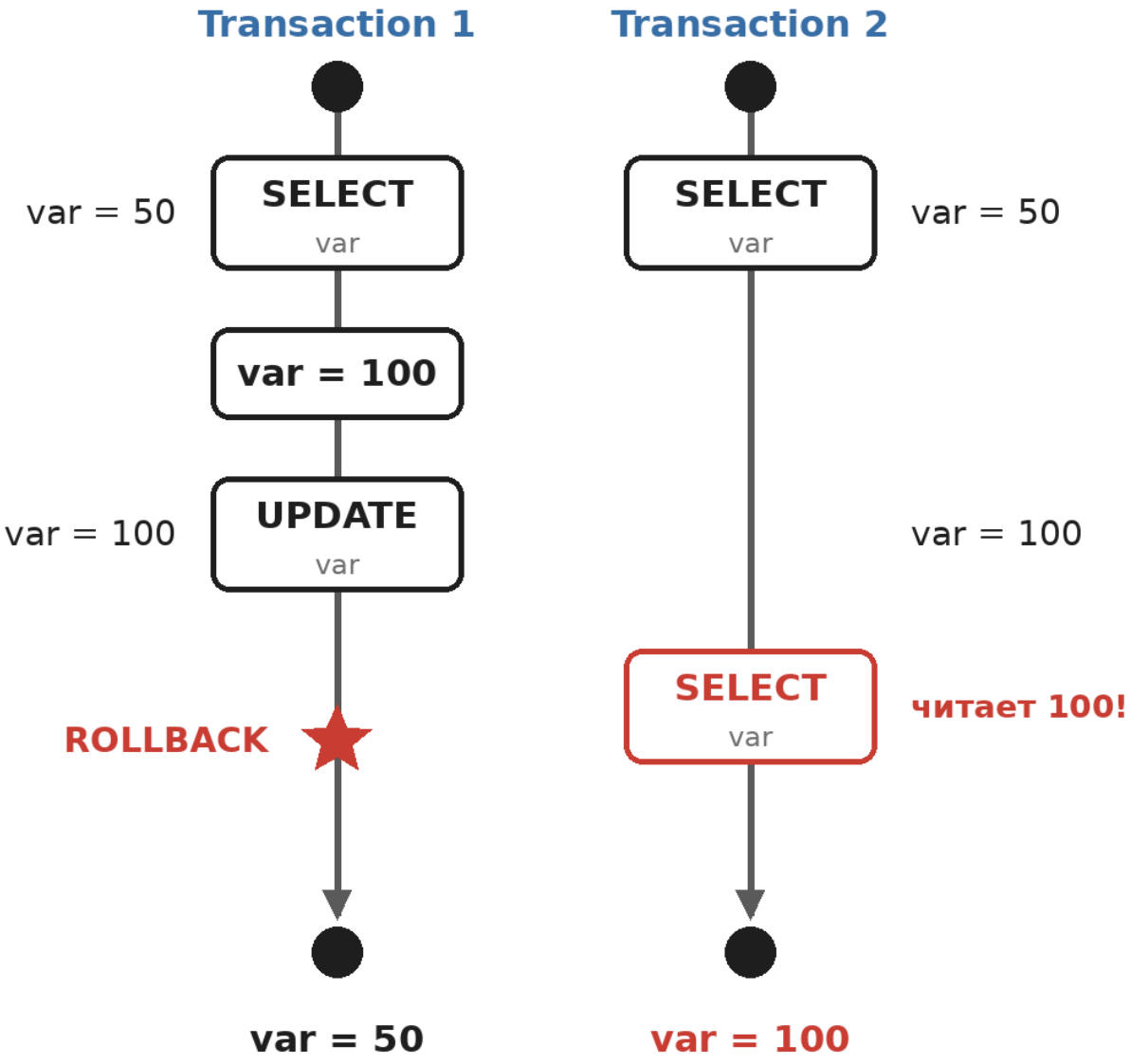
- `READ COMMITTED`
- `REPEATABLE READ`
- `SERIALIZABLE`

Применение транзакций:

- **Архивация и удаление данных:** использование временных таблиц или сохранение данных с последующим их удалением.
- **Работа с чувствительными данными:** внесение изменений и возможность их отката до выполнения `COMMIT`.
- **Временные точки отката:** использование `SAVEPOINT` для отката к определенной точке, не отменяя всю транзакцию.

DIRTY READS

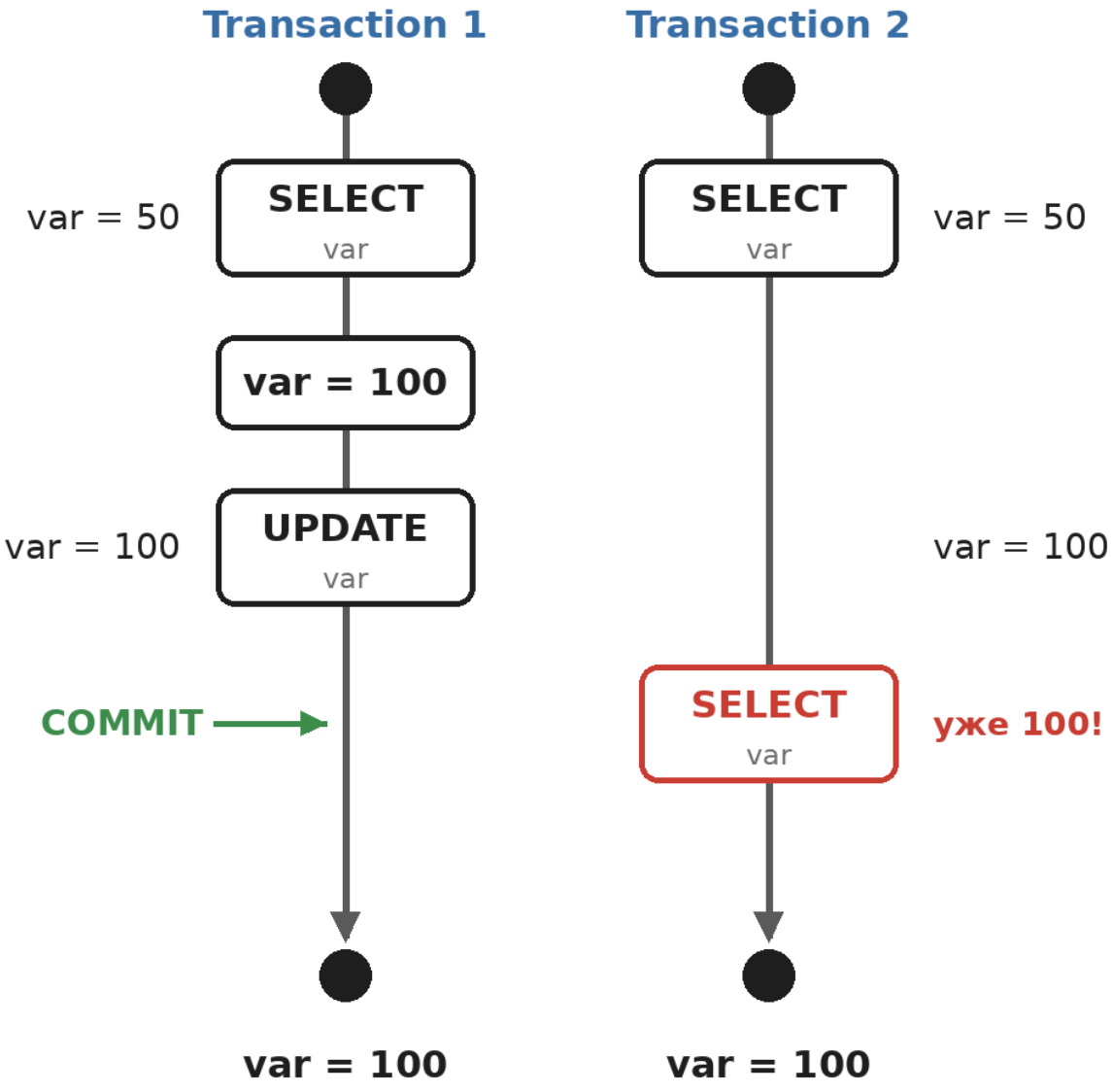
«Грязное» чтение



T2 прочитала значение, которое так и не было зафиксировано

NON-REPEATABLE READS

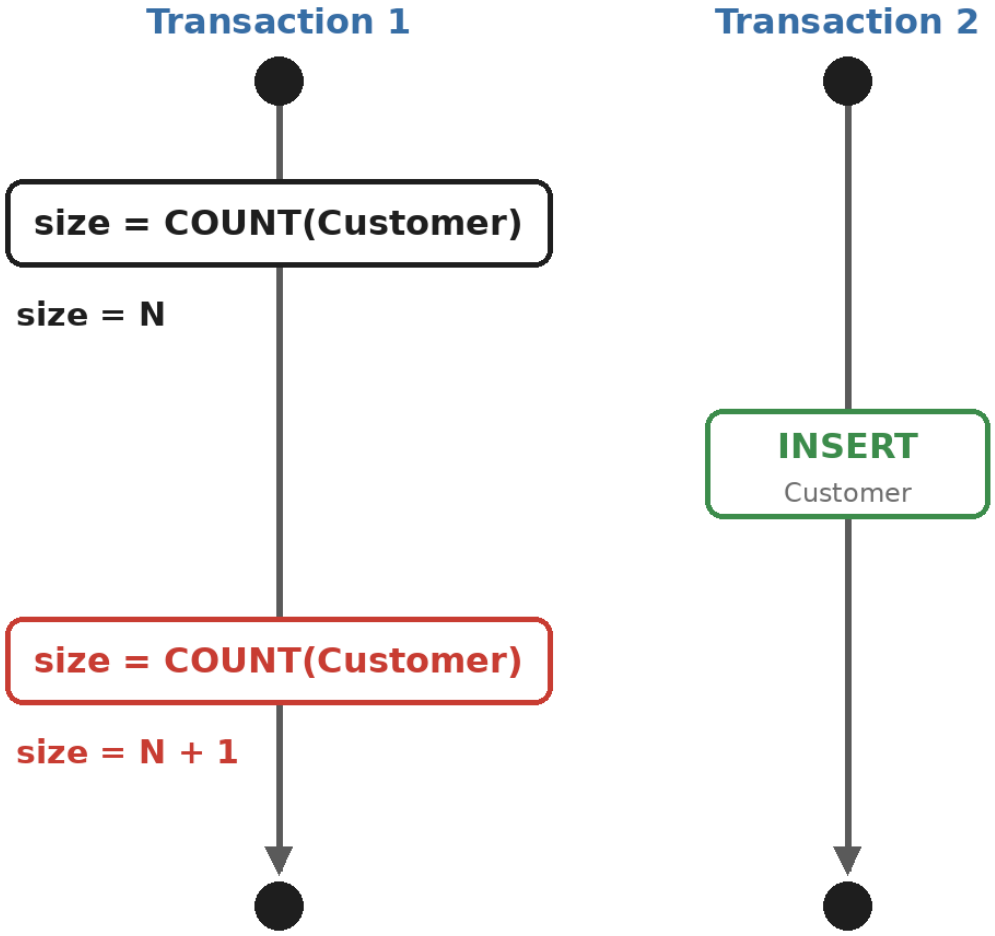
Неповторяемое чтение



Повторный SELECT в T2 вернул другое значение той же строки

PHANTOM READS

Фантомное чтение



Повторный запрос в T1 увидел новую строку, вставленную T2

SERIALIZATION ANOMALY

Аномалия сериализации

Таблица mytab

class	value
1	10
1	20
2	100
2	200

class 1: 10 + 20 = 30

class 2: 100 + 200 = 300

Транзакция A	Транзакция B
`SELECT SUM(value) FROM mytab` `WHERE class = 1;  -- 30`	`SELECT SUM(value) FROM mytab` `WHERE class = 2;  -- 300`
`INSERT INTO mytab` `VALUES (30, 2);`	`INSERT INTO mytab` `VALUES (300, 1);`
`COMMIT;`	`COMMIT;`

Каждая транзакция читает данные, которые меняет другая. При любом последовательном порядке одна увидела бы вставку другой — параллельно результат не совпадает ни с одним.

Итог: A вставит `(30, 2)`, B вставит `(300, 1)`, потому что ни одна не видит вставку другой.

При поочерёдном выполнении вторая транзакция увидела бы вставку первой и получила бы **330**, а не 30 или 300. Параллельный результат не совпадает ни с одним последовательным порядком, это и есть аномалия.

На `SERIALIZABLE` PostgreSQL такого не допустит: одна из транзакций упадёт с ошибкой сериализации (SQLSTATE `40001`), и её нужно повторить.

---

14. Безопасность

Роли и пользователи

- **Роль** — совокупность прав доступа в базе данных. Может быть с логином (пользователь) или без.
- **Пользователь** — роль с логином и паролем.

- Роли могут включать другие роли, создавая иерархию прав доступа.
- Роли создаются на уровне экземпляра сервера и не привязаны к конкретной базе данных.

Уровни безопасности

1. **Экземпляр:** управление подключениями, аутентификацией и созданием баз данных.
2. **База данных:** права на подключение и управление объектами.
3. **Схемы:** доступ к созданию и удалению схем.
4. **Таблицы:** операции CRUD.
5. **Колонки:** операции над отдельными столбцами.
6. **Строки:** доступ на уровне строк (row-level security).

Создание и управление ролями

- **Создание роли:**

```
CREATE ROLE role_name [WITH LOGIN PASSWORD 'password'];
```

- **Права роли:**
  - `LOGIN`, `SUPERUSER`, `CREATEDB`, `CREATEROLE`, `REPLICATION`.
- **Список ролей:**

```
SELECT rolname FROM pg_roles;
```

Лучшие практики

- Работать не из-под роли `SUPERUSER`, а через специально созданных пользователей с минимально необходимыми правами.
- Создавать роли для различных сценариев использования (например, `read_only`, `read_write`).
- Минимизировать права и давать доступы только при необходимости.

Управление доступом

- **Доступ к таблицам:**

```
GRANT SELECT, INSERT ON table_name TO role_name;
```

- **Доступ к колонкам:**

```
GRANT SELECT (column_name) ON table_name TO role_name;
```

- **Управление доступом на уровне строк:**

```
ALTER TABLE table_name ENABLE/DISABLE ROW LEVEL SECURITY;
CREATE POLICY policy_name ON table_name FOR SELECT TO role_name
USING (condition);

DROP POLICY IF EXISTS policy_name ON table_name;
```

Удаление ролей и прав

- Сначала необходимо:
  - Изъять права:
    ```
    REVOKE ALL PRIVILEGES ON table_name FROM role_name;
    ```
  - Переназначить объекты:
    ```
    ALTER TABLE table_name OWNER TO new_role;
    ```
- Удаление роли:
  ```
  DROP ROLE IF EXISTS role_name;
  ```

Важные моменты

- **Row-Level Security (RLS):** после включения RLS доступ к данным блокируется по умолчанию, необходимо явно настраивать политики доступа.
- **DROP OWNED BY:** удаляет объекты, принадлежащие роли.